

Microservices & Serverless Computing

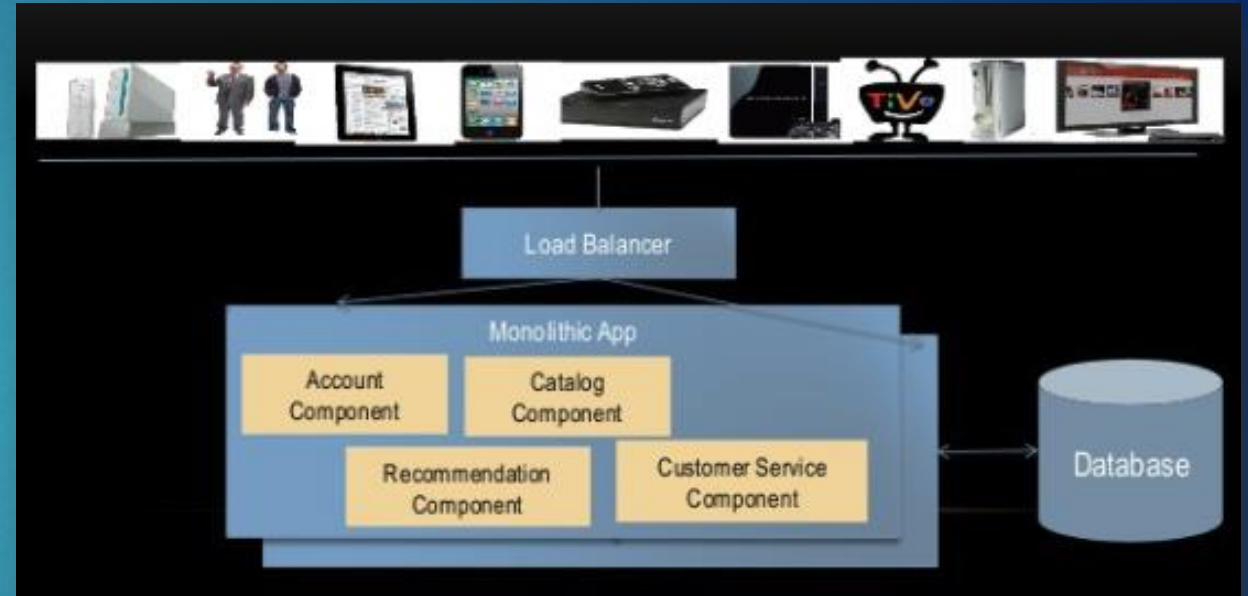
Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



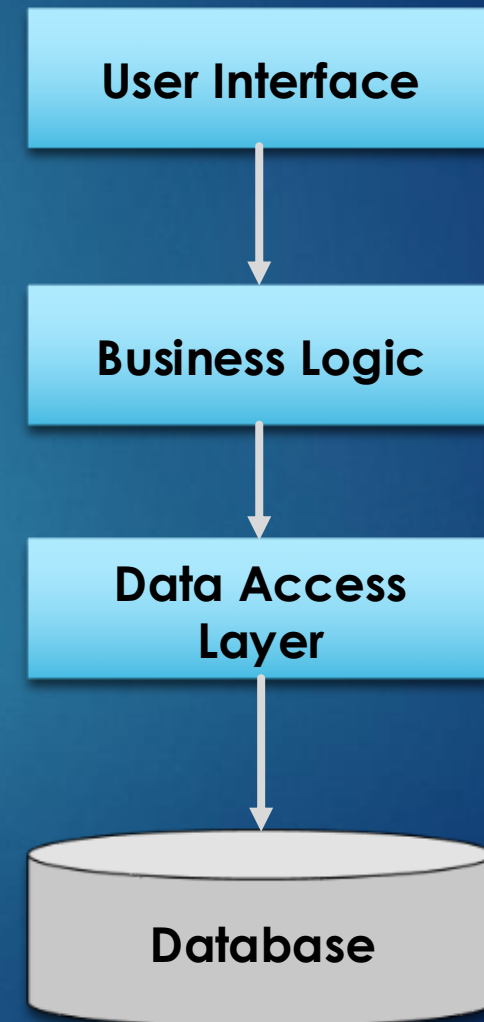
Netflix - 2008

- ▶ In 2008, Netflix was running relational databases in its own data center when disaster struck
- ▶ A data center failure shut the entire service down and stopped DVD shipments for three days
- ▶ Netflix was growing fast, and the thousands of videos and tens of millions of customers was already generating an enormous quantity of data
- ▶ The company would struggle to rack the servers in their own data centers fast enough to handle the ever-growing volumes, but the cloud would let them add thousands of virtual servers and petabytes of storage within minutes
- ▶ Before they moved to the cloud, they had to take a step back and rearchitect their Monolithic Architecture



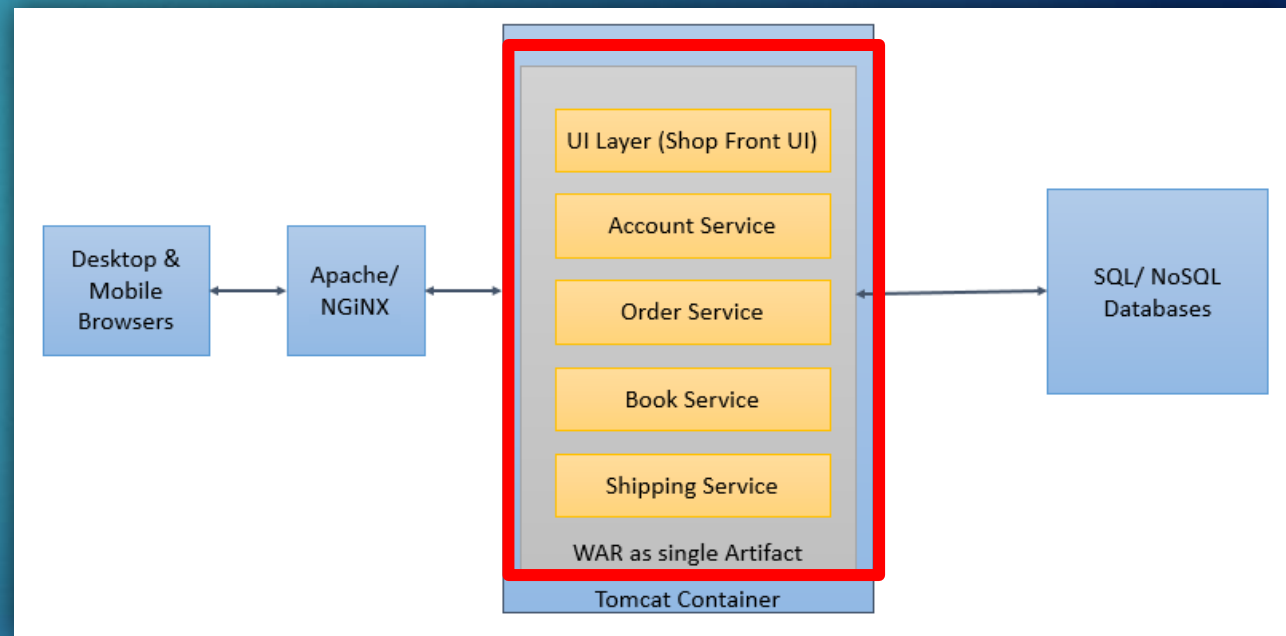
What is a Monolithic Architecture?

- ▶ A Monolithic Architecture is the traditional unified model for the design of a software program
- ▶ Monolithic software is designed to be self-contained where components of the program are interconnected and interdependent
- ▶ In this tightly-coupled architecture, each component and its associated components must be present in order for code to be executed or compiled



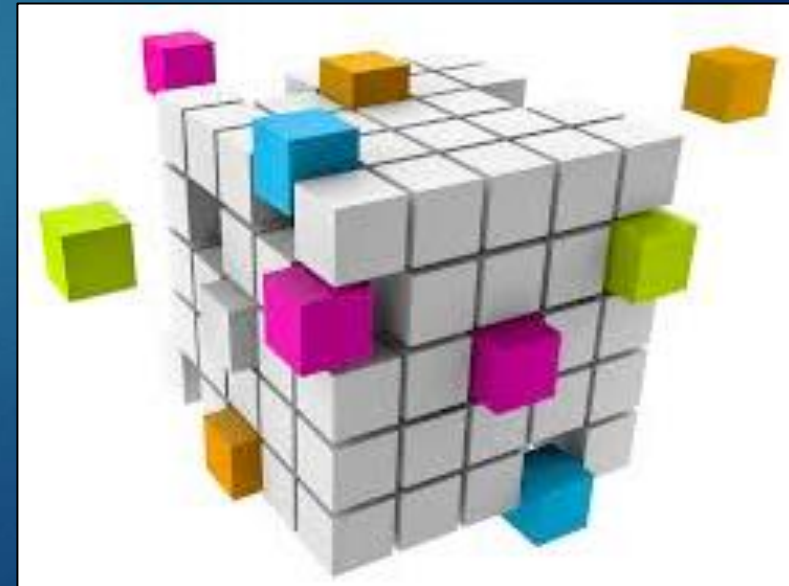
Monolithic Architecture - Example

- ▶ All modules of this traditional web application, such as **Shop Front UI**, **Account Service**, **Order Service**, **Book Service**, and **Shipping Service**, are single artifact (WAR file) in the Tomcat container
- ▶ This monolithic application has a massive codebase that includes all modules
- ▶ If you introduce new modules to this application, you must make changes to the existing code and then redeploy the WAR file to the Tomcat container



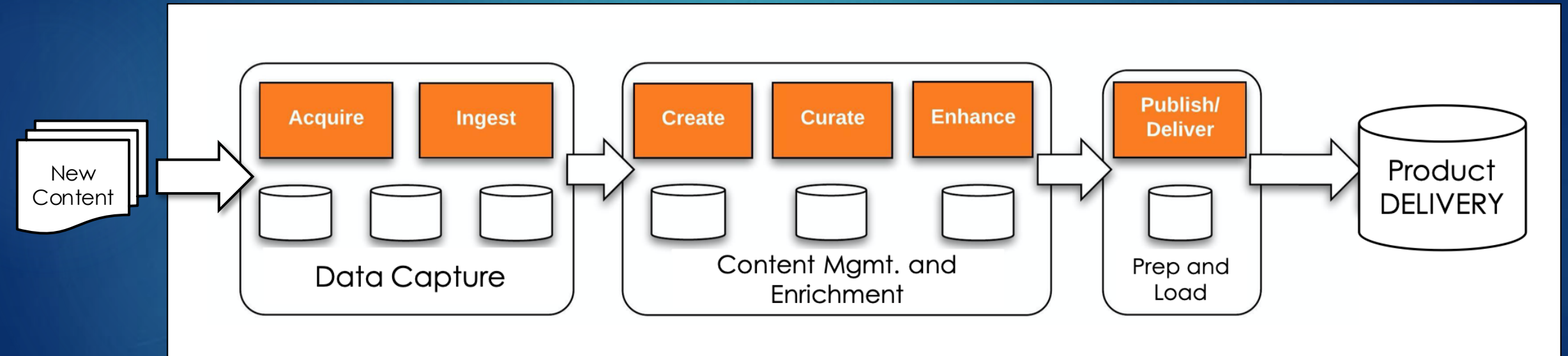
Monolithic Architecture - Challenges

- ▶ **Inflexible** – Monolithic applications cannot be built using different technologies
- ▶ **Unreliable** – Even if one feature of the system does not work, then the entire system does not work
- ▶ **Unscalable** – Applications cannot be scaled easily since each time the application needs to be updated, the complete system has to be rebuilt
- ▶ **Blocks Continuous Development** – Many features of the applications must be built and deployed at the same time
- ▶ **Slow Development** – Development in monolithic applications take lot of time to be built since each and every feature has to be built one after the other
- ▶ **Not Fit For Complex Applications** – Features of complex applications have tightly coupled dependencies



Monolithic Architecture – Rearchitect?

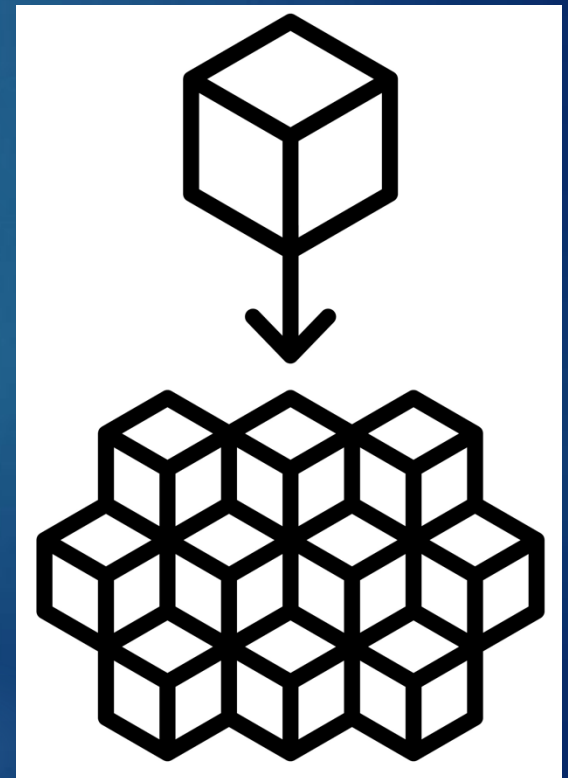
- ▶ Consider the following monolithic publishing system, which runs completely on premise
- ▶ Many of these services use legacy technologies, many of which are not clearly understood



- ▶ How would you rearchitect this entire system to run on the cloud?

Monolithic Architecture – Considerations when moving to the cloud

- ▶ A “lift and shift” to the cloud would be more costly and likely would have other limitations (similar to Netflix)
- ▶ For starters, you would decompose the problem into smaller services, which would allow more teams to work on this
 - ▶ Services should be reusable where it makes sense
- ▶ Although you would prefer to use a single programming language, this is not a hard fast requirement
- ▶ As this will all be new, you need to be able to respond very quickly to fixes and/or modifications and release quickly
- ▶ This needs to be scalable but also cost effective
- ▶ This is where microservice can help!



What are Microservices?

- ▶ Microservices is an architectural style that structures an application as a collection of small *autonomous* services modeled around a business domain
- ▶ Each of these services is responsible for discrete task (aka fine-grained) and can communicate with other services through simple APIs
- ▶ A microservices architecture enables developers to build loosely coupled services which can be developed, deployed, and maintained independently



Microservice Key Features

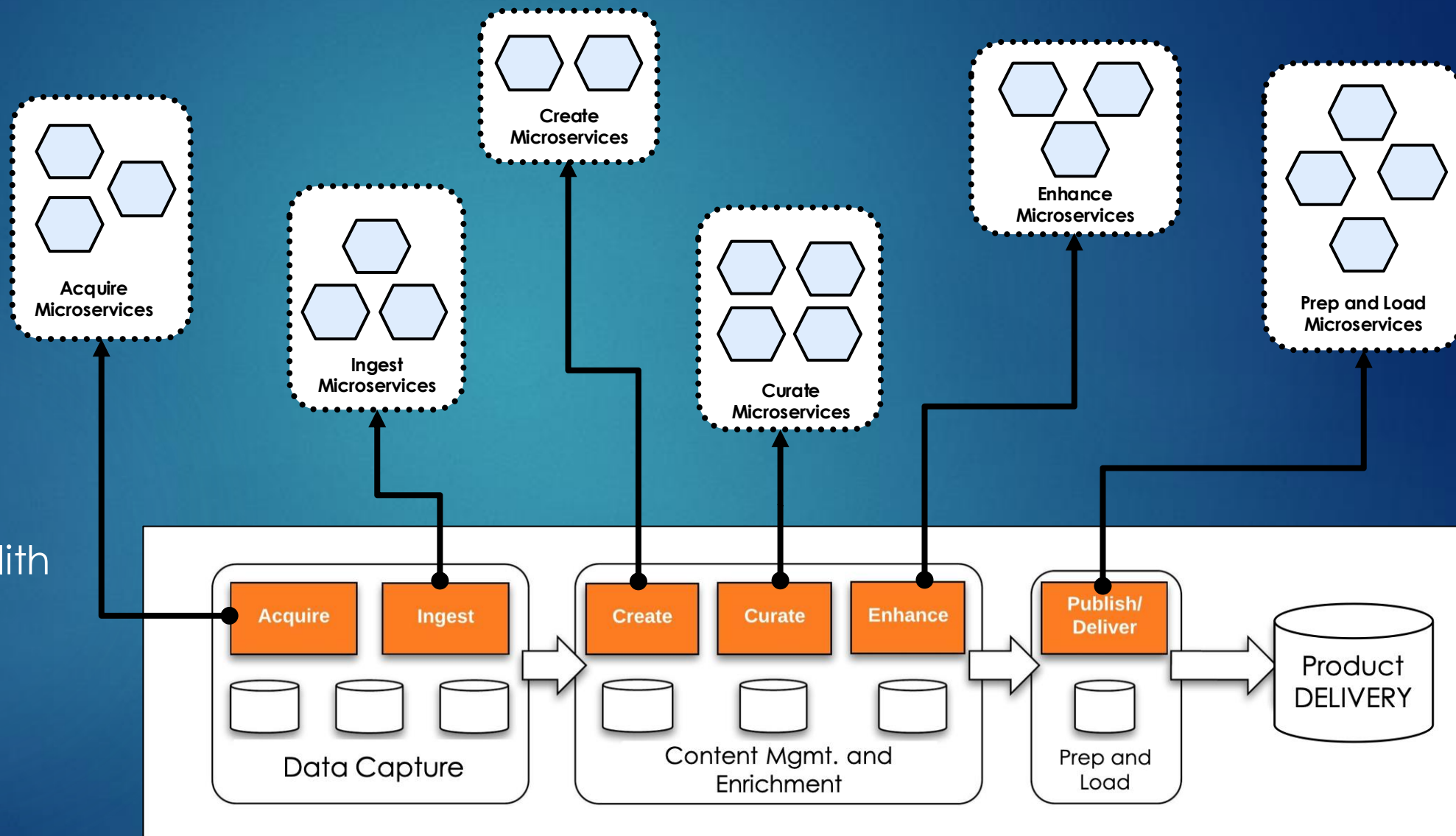
- ▶ **Decoupling** – Services within a system are largely decoupled
- ▶ **Componentization** – Microservices are treated as independent components that can be easily replaced and upgraded
- ▶ **Business Capabilities** – Microservices are very simple and focus on a single capability
- ▶ **Continuous Delivery** – Allows frequent releases of software, through systematic automation of software creation, testing, and approval
- ▶ **Decentralized Governance** – The focus is on using the right tool for the right job. There is no standardized or technology pattern so developers have the freedom to choose the best useful tools to solve their problems.



Monolithic to Microservices

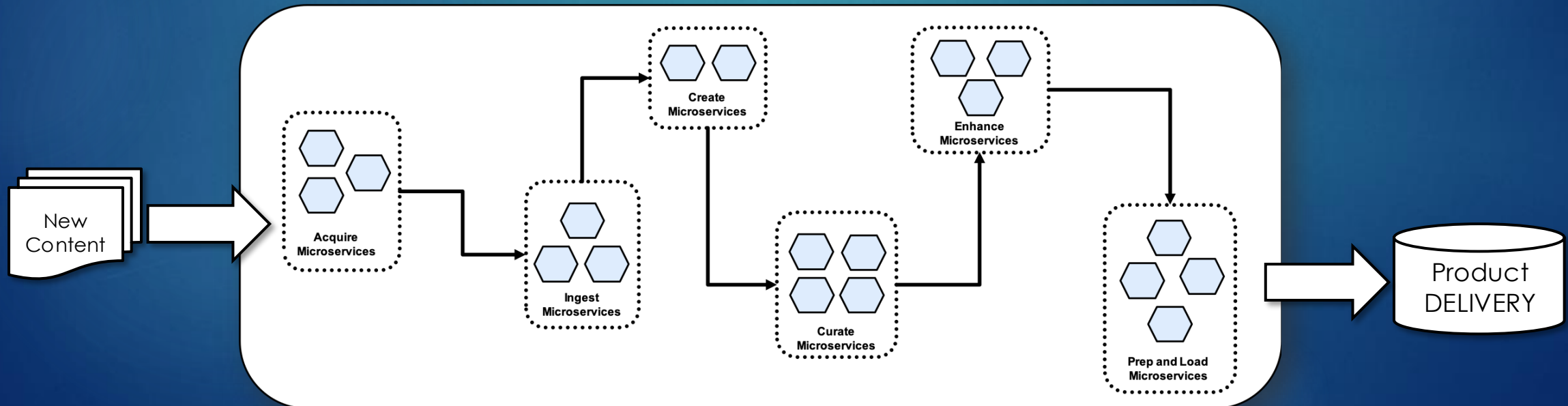
10

- Split the monolith to build a microservices architecture



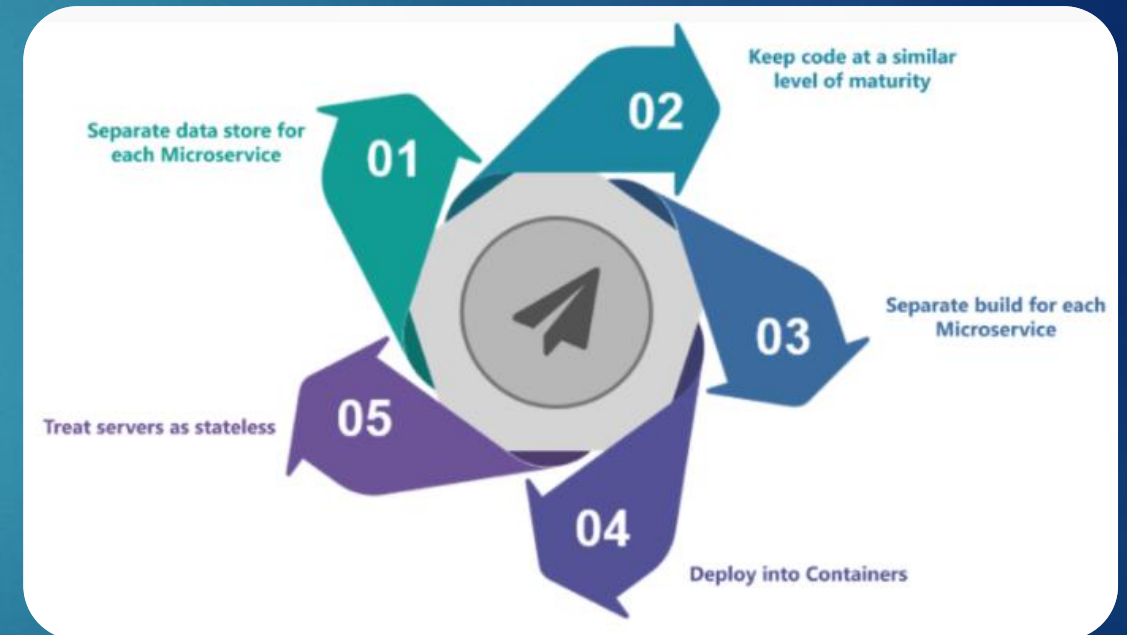
Monolithic to Microservices

- ▶ We now have our publishing system converted to microservices, but some questions remain:
 - ▶ How do we create a microservice?
 - ▶ How do microservices communicate with each other?
 - ▶ How can we run these as cost efficiently as possible?



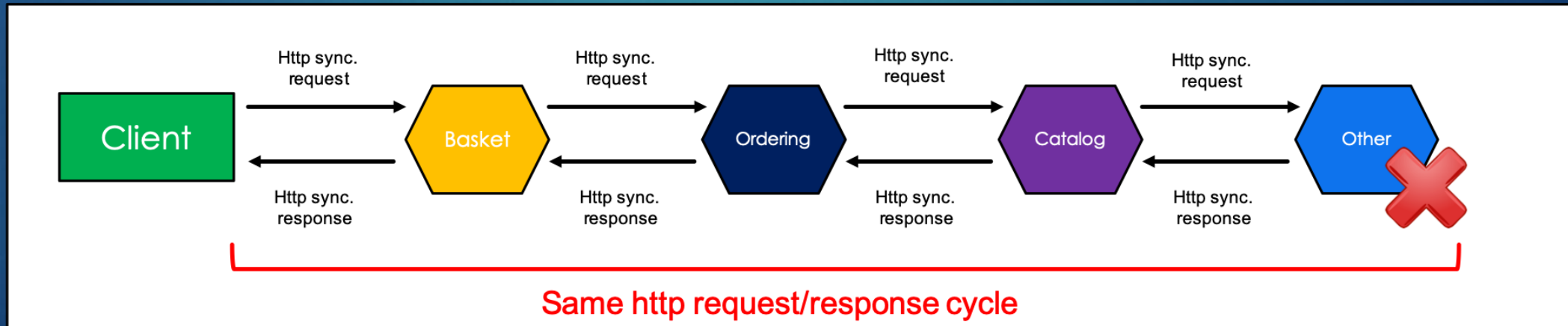
How do we create a Microservice?

- ▶ Remember, microservices are an architectural style
 - ▶ It's a matter of following best practices
- ▶ Using containers is the most ideal option for microservices
 - ▶ Very lightweight
 - ▶ Faster initialization and execution
 - ▶ They can start and stop independently of each other
- ▶ How do we manage all these microservices?
 - ▶ Orchestration tool (ECS, Kubernetes, etc.)



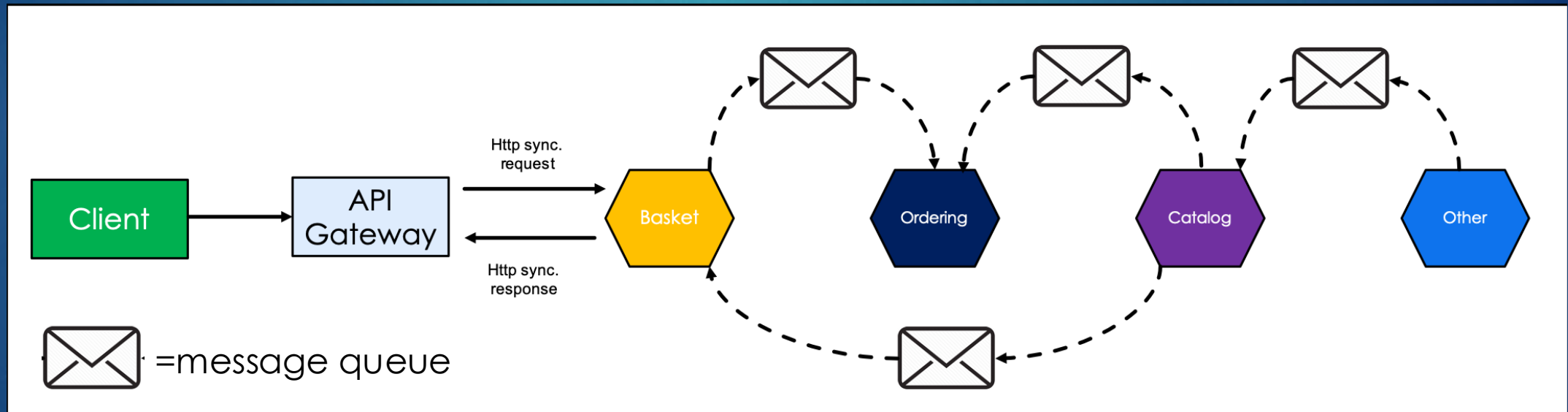
How do Microservices communicate with each other? 13

- ▶ RESTful API makes it easy to build a loosely coupled Microservices
- ▶ However, be careful about chaining too many microservices together as this can be inefficient
- ▶ Why? As you add more synchronous dependencies between microservices, such as a query requests, the worse the performance and resiliency



How do Microservices communicate with each other? 14

- ▶ Using an asynchronous protocol means that the client code or message sender usually doesn't wait for a response
- ▶ One way this can be achieved is where microservices can subscribe to a message queue



What are Messaging Queues?

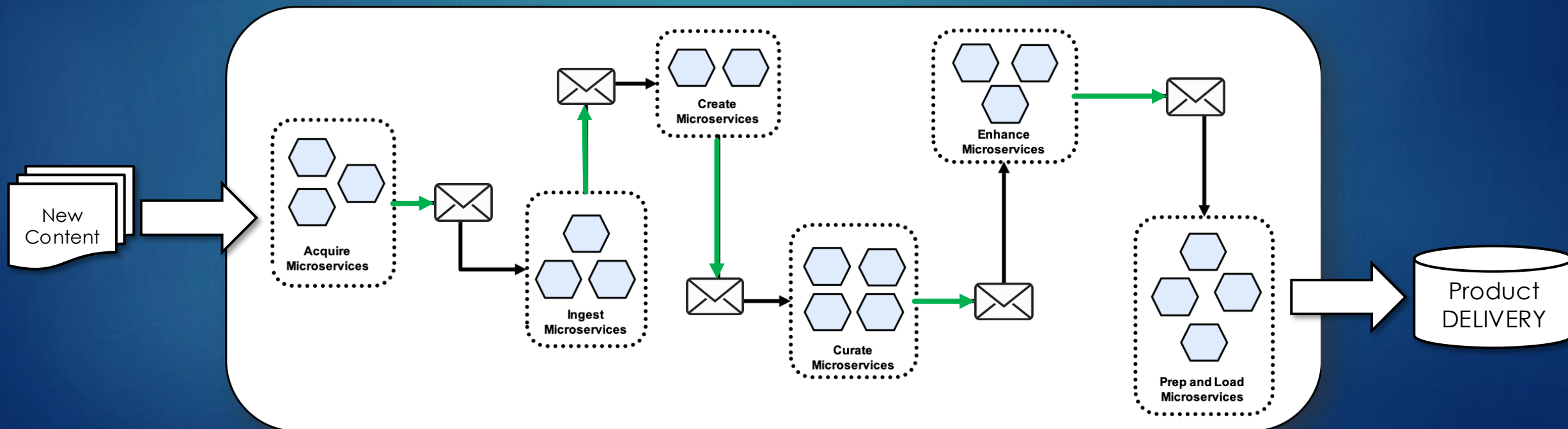
- ▶ Message Queues provide communication and coordination for distributed applications
- ▶ The messages are usually small, and can be things like requests, replies, error messages, or just plain information
- ▶ To send a message, a component called a producer adds a message to the queue
- ▶ The message is stored on the queue until another component called a consumer retrieves the message and does something with it



- ▶ Amazon's Simple Queue Service (SQS) offers a secure, durable, and available hosted queue that lets you integrate and decouple distributed software systems and components

Back to our Example

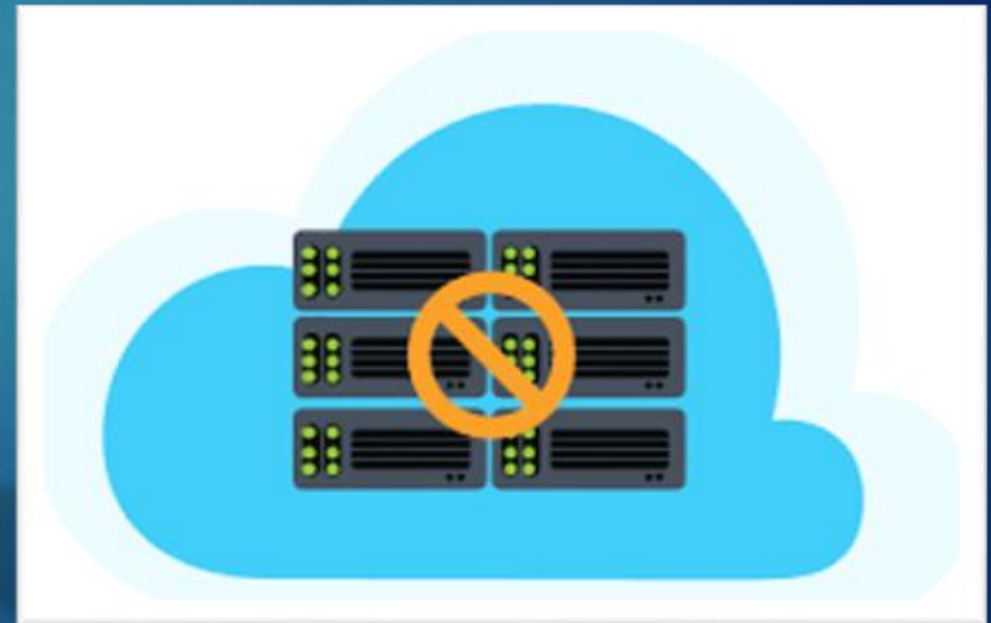
- ▶ Asynchronous microservice integration enforces microservice's autonomy
- ▶ The fewer communications between microservices, the better



How can we run these as cost efficiently as possible?

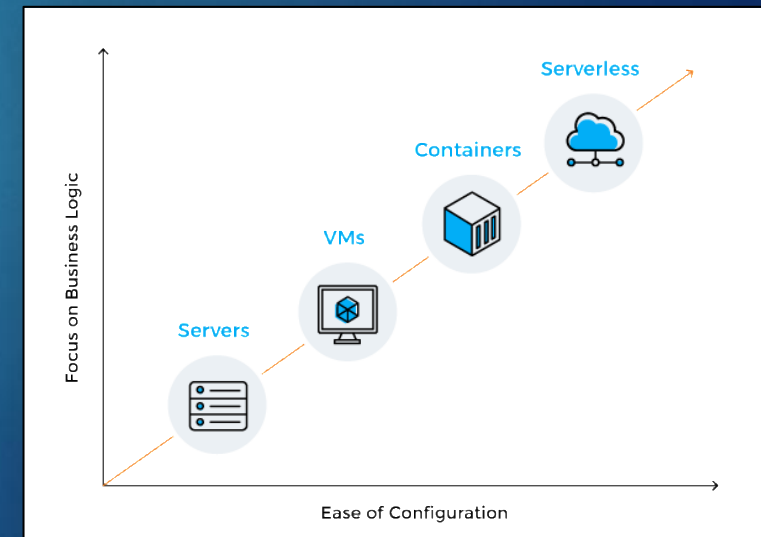
17

- ▶ Running on EC2 servers allows us to use Auto-scaling, which can help to save costs
- ▶ But we are still paying for the EC2 servers, which might be more than we need for processes that run very quickly or very little resources
- ▶ I may have thousands and thousands of microservices running
- ▶ I don't want to think about servers
- ▶ I would prefer to pay only for only the computing I need
- ▶ Solution: Serverless computing



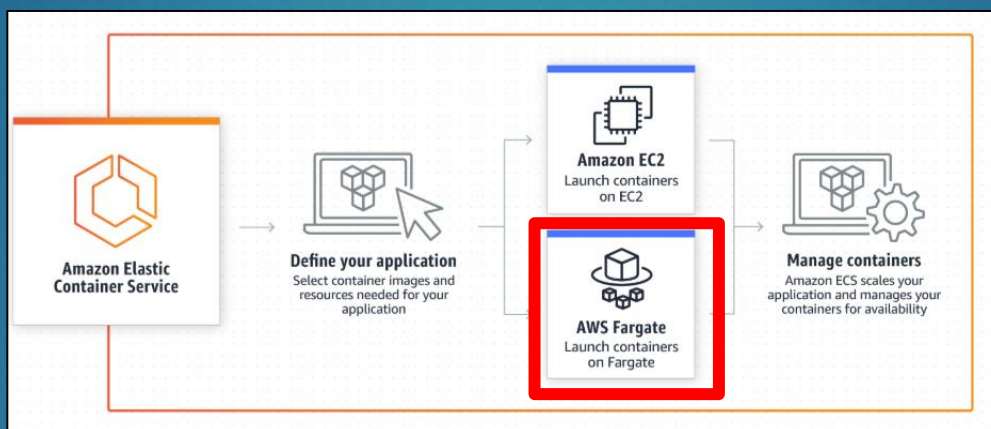
What is Serverless Computing?

- ▶ Serverless Computing is a method of providing backend services on an as-used basis
- ▶ Pricing is based on the actual amount of resources consumed by an application, rather than on pre-purchased units of capacity
- ▶ A Serverless provider allows users to write and deploy code without the hassle of worrying about the underlying infrastructure
- ▶ Although called serverless, physical servers are still used but developers do not need to be aware of them



Serverless Computing - Examples

- ▶ For our Container Orchestration activity, we talked about AWS Fargate, which is a serverless compute engine for containers



AWS Fargate removes the need to provision and manage servers, lets you specify and pay for resources per application

- ▶ Another serverless example is compute service like AWS Lambda
 - ▶ This is a compute service that lets you run code without provisioning or managing servers
 - ▶ This is sometimes referred to as FaaS (Functionality as a Service) and all cloud providers support similar functionality

Lambda vs. Fargate vs. EC2

20



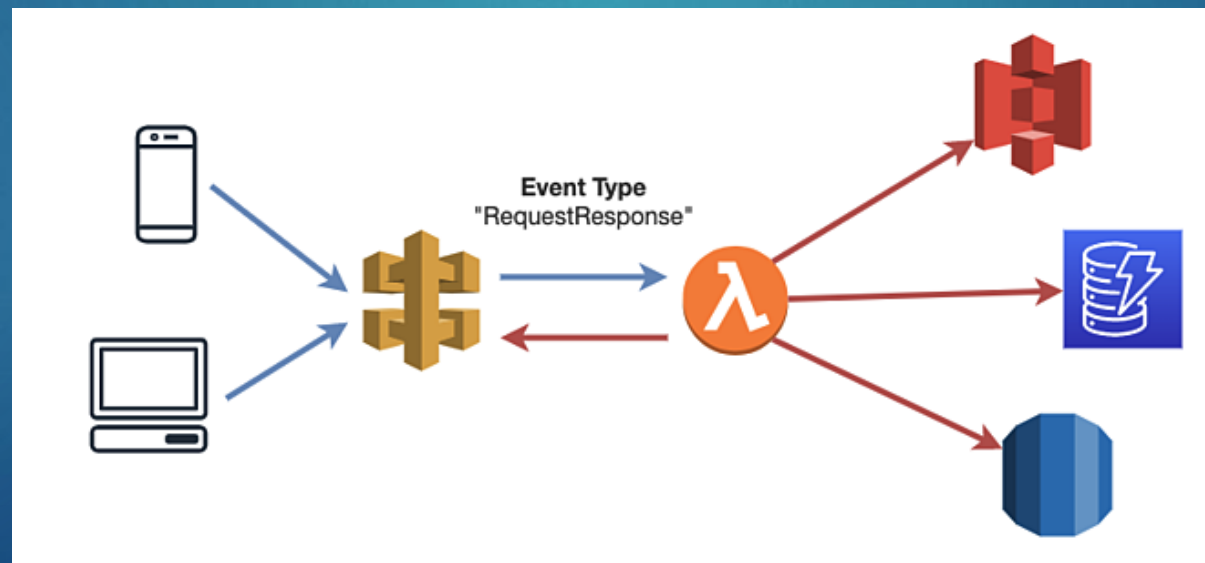
AWS Lambda - Overview



- ▶ Built-in fault tolerance
- ▶ Automatically can scale to support the rate of incoming requests without requiring you to configure anything
- ▶ Pay per execution only based on 2 factors: Requests and Duration
- ▶ Requests
 - ▶ Lambda counts a request each time it starts executing in response to an event notification or invoke call
 - ▶ Each request costs \$0.0000002 (or 50,000 calls for .01)
- ▶ Duration
 - ▶ Calculated from the time your code begins executing until it returns or otherwise terminates, rounded up to the nearest 100ms
 - ▶ The price is not charged per second but per GB-second, which is the duration in seconds multiplied by the maximum memory size in GB
 - ▶ Every GB-second costs \$0.0000166667

AWS Lambda - Overview

- ▶ They can be invoked synchronously or asynchronously through some sort of trigger. Some common examples:
 - ▶ When a message appears on a queue, trigger a lambda to retrieve it
 - ▶ When a file appears in a file location, trigger a lambda to take a specific action
 - ▶ When an API on the API Gateway is called, it can trigger a lambda to do something like accessing a database



API Gateway & Lambdas

23

Resources

Create resource

- /
- /mma
 - /v1
 - /({rail})
 - /batch POST
 - /fields GET
 - /recurring POST
 - /transaction POST
 - /about GET

/mma/v1/({rail})/fields - GET - Method execution

API actions ▼ Deploy API

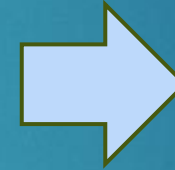
Update documentation Delete

ARN: arn:aws:execute-api-east-1:416638788683:sbxmhn2oa4/*/GET/mma/v1/({rail})/fields
Resource ID: i8lshn

Client → Method request → Integration request → Undefined integration
← Method response ← Integration response

Undefined integration
Define an integration to create your method.

Edit integration



Edit integration request

Method details

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **Mock**
Generate a response based on API Gateway mappings and transformations.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.

☒ Lambda proxy integration

Send the request to your Lambda function as a structured event.

Lambda function

Provide the Lambda function name or alias. You can also provide an ARN from another account.

us-east-1 ▼

Choose a Lambda function or enter its ARN

☒ Default timeout

The default timeout is 29 seconds.

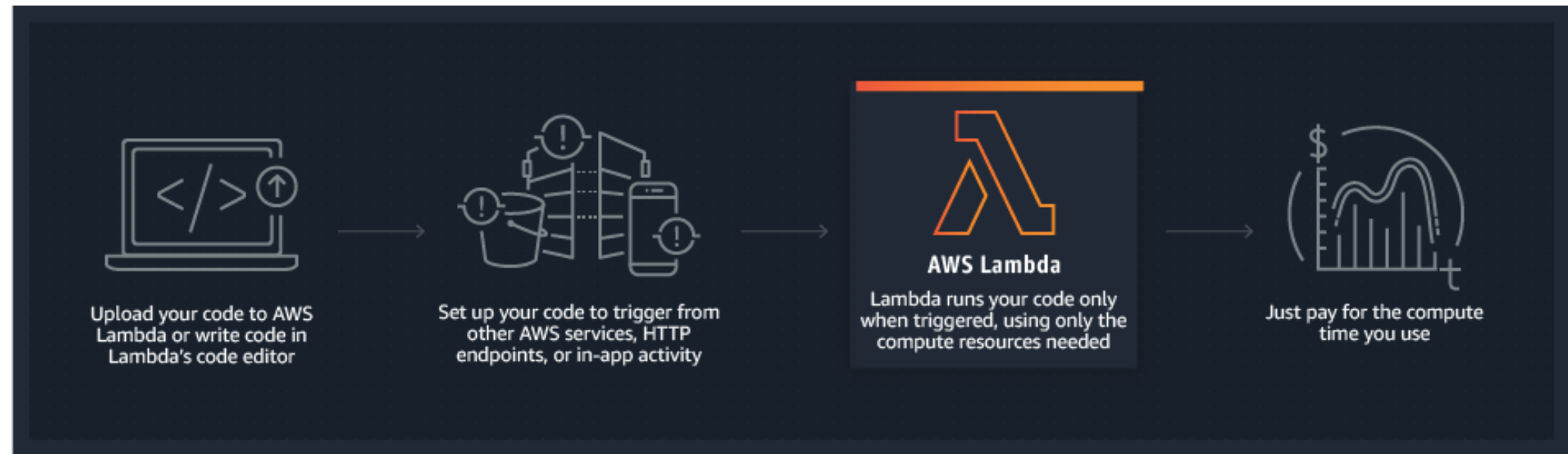
Request body passthrough

- ☒ When there are no templates defined (recommended)
- ☐ When no template matches the request content-type header
- ☐ Never

Specify your
Lambda
function here

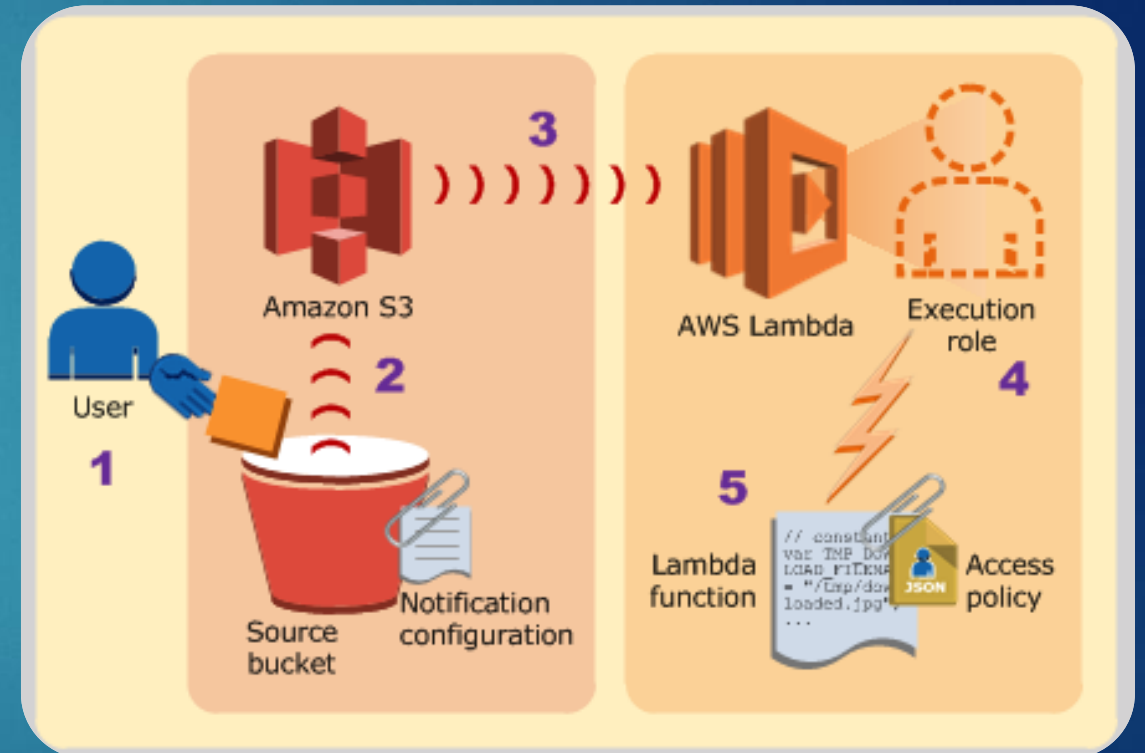
AWS Lambda – How does it work?

- ▶ They can be created in a number of languages like Python, Nodejs, Java, Go, Ruby, and C#



Lambda Function – S3 Example

- ▶ The flow illustrates a simple example using S3:
 1. User uploads an object to an S3 bucket (object-created event)
 2. Amazon S3 detects the object-created event
 3. Amazon S3 “triggers” a Lambda function that is specified in the bucket notification configuration
 4. AWS Lambda executes the Lambda function by assuming the execution role that you specified at the time you created the Lambda function
 5. The Lambda function executes



Lambda Function – S3 Example

- Upon a triggered event, this is an example of the event object gets passed to your Lambda function

```
{
  "Records": [
    {
      "eventVersion": "2.1",
      "eventSource": "aws:s3",
      "awsRegion": "xx-xxx-x",
      "eventTime": "2020-04-08T19:36:34.075Z",
      "eventName": "ObjectCreated:Put",
      "userIdentity": {
        "principalId": "AWS:POWERUPCLOUD:powerup@powerupcloud.com"
      },
      "requestParameters": {
        "sourceIPAddress": "XXX.XX.XXX.XX"
      },
      "responseElements": {
        "x-amz-request-id": "POWERUPCLOUD",
        "x-amz-id-2": "POWERUPCLOUD/POWERUPCLOUD/POWERUPCLOUD/POWERUPCLOUD/POWERUPCLOUD/POWERUPCLOUD/POWERUPCLOUD/POWERUPCLOUD"
      },
      "s3": {
        "s3SchemaVersion": "1.0",
        "configurationId": "powerup24-powerup-powerup-powerup",
        "bucket": {
          "name": "source-test-bucket-006",
          "ownerIdentity": {
            "principalId": "POWERUPCLOUD"
          },
          "arn": "arn:aws:s3:::source-test-bucket-006"
        },
        "object": {
          "key": "AjayMuralidhar/2020-04-06/12/my-tasks.txt",
          "size": 20,
          "eTag": "1853ea0cebd1e10d791c9b2fcb8cc334",
          "sequencer": "005E8E27C31AEBFA2A"
        }
      }
    }
  ]
}
```

The diagram highlights the 's3' object within the event record. A red rounded rectangle encloses the 's3' object. Two callout boxes with arrows point to specific fields: 'S3 Bucket name' points to the 'name' field of the 'bucket' object, and 'New object (file) name' points to the 'key' field of the 'object' object.

- The event object differs in structure and contents, depending on which event source created it
- The contents of the event parameter include all the data and metadata your Lambda function needs to drive its logic
- In this example, the event created by Amazon S3 when a new object is created will include details about the bucket and the new object

Lambda Function – S3 Example

- ▶ This Lambda function is called (triggered) when a file is uploaded to a source bucket
- ▶ It copies the file from the source bucket to another bucket (destination)

```
import json
import boto3

# boto3 S3 initialization
s3_client = boto3.client("s3")

def lambda_handler(event, context):
    destination_bucket_name = 'destination-test-bucket'

    # event contains all information about uploaded object
    print("Event :", event)

    # Bucket Name where file was uploaded
    source_bucket_name = event['Records'][0]['s3']['bucket']['name']

    # Filename of object (with path)
    file_key_name = event['Records'][0]['s3']['object']['key']

    # Copy Source Object
    copy_source_object = {'Bucket': source_bucket_name, 'Key': file_key_name}

    # S3 copy object operation
    s3_client.copy_object(CopySource=copy_source_object, Bucket=destination_bucket_name, Key=file_key_name)

    return {
        'statusCode': 200,
        'body': json.dumps('Hello from S3 events Lambda!')
    }
```

← boto3 is a Python SDK for AWS

← Destination bucket

← Source bucket from event object

← File name (object) that was uploaded

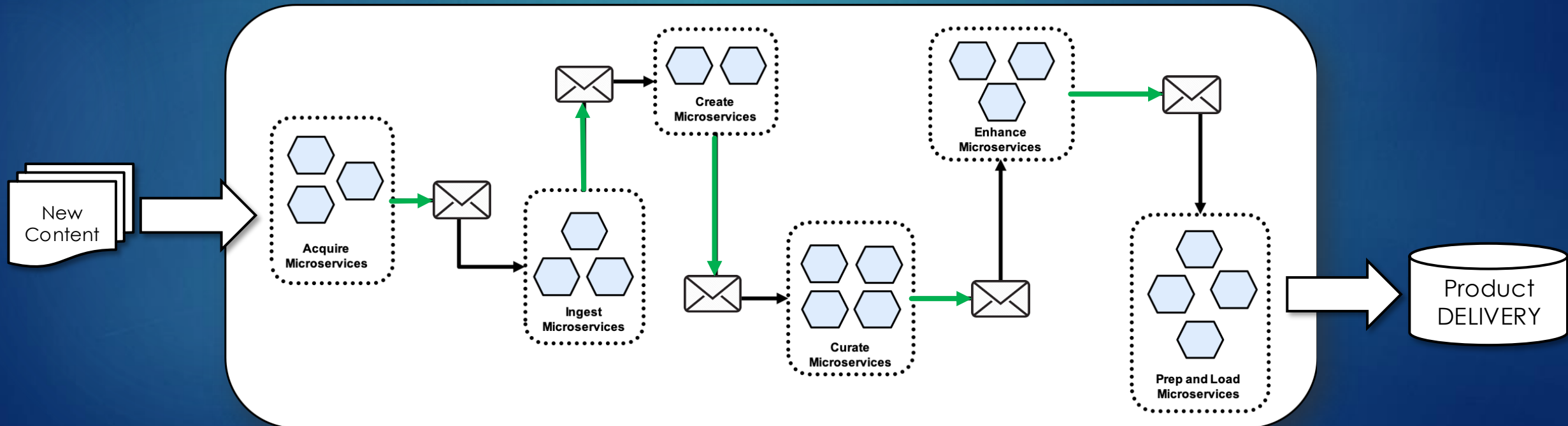
← Source = bucket + file name

← Copy source to destination bucket

Context → The data about the execution environment of the function (e.g. function name, log group, etc.)

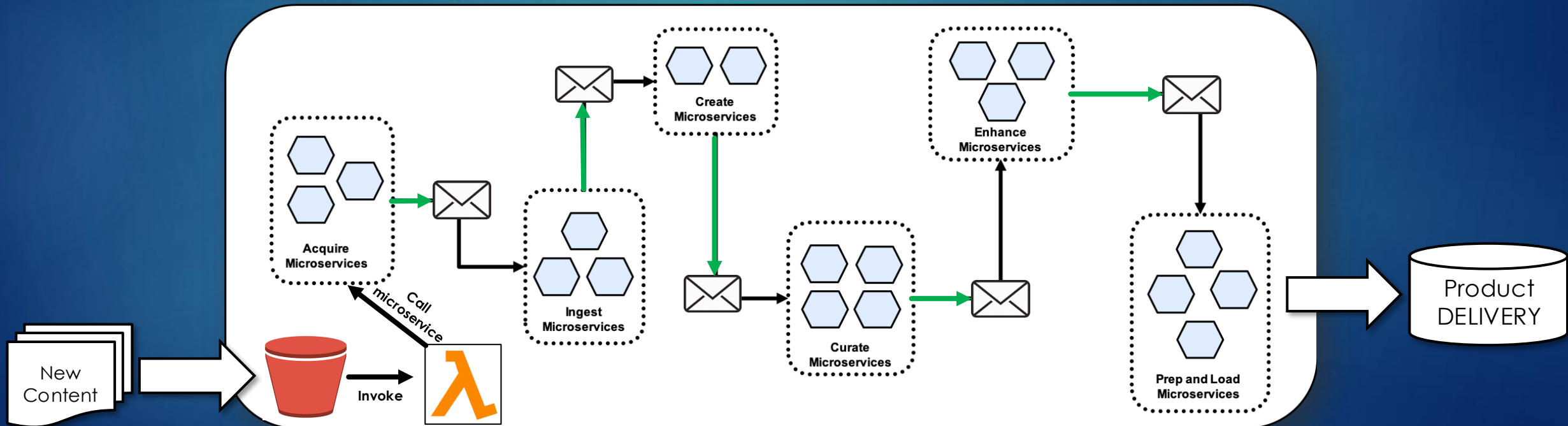
Back to our Example

- ▶ Our microservices are available but how are they triggered?
- ▶ A simple Lambda function would work



Back to our Example

- ▶ Our microservices are available but how are they triggered?
- ▶ A simple Lambda function would work



Lambda Function – Call a Microservice

- ▶ This Lambda function is called (triggered) when a file is uploaded to a source bucket
- ▶ It will pass the bucket and file name to a microservice

```
import json
import boto3
import requests

# boto3 S3 initialization
s3_client = boto3.client("s3")

def lambda_handler(event, context):

    # Bucket Name where file was uploaded
    source_bucket_name = event['Records'][0]['s3']['bucket']['name']

    # Filename of object (with path)
    file_key_name = event['Records'][0]['s3']['object']['key']

    # Copy Source Object
    copy_source_object = {'Bucket': source_bucket_name, 'Key': file_key_name}

    # defining the api-endpoint
    API_ENDPOINT = http://reuters.com/v1/my-microservice

    # sending post request and saving response as response object
    r = requests.post(url = API_ENDPOINT, data = copy_source_object)

    return {
        'statusCode': 200,
        'body': json.dumps('Hello from S3 events Lambda!')
    }
```

Specify URL to microservice

Call microservice with source bucket and file name

Lambdas – Pros/Cons

► Pros

- No server management as Lambdas automatically run with provisioning any servers
- Lambda scales automatically in response to the number of incoming events or the traffic to your application
- Cost-efficient as you pay only for the compute time your code consumes
- Integrates with other AWS services

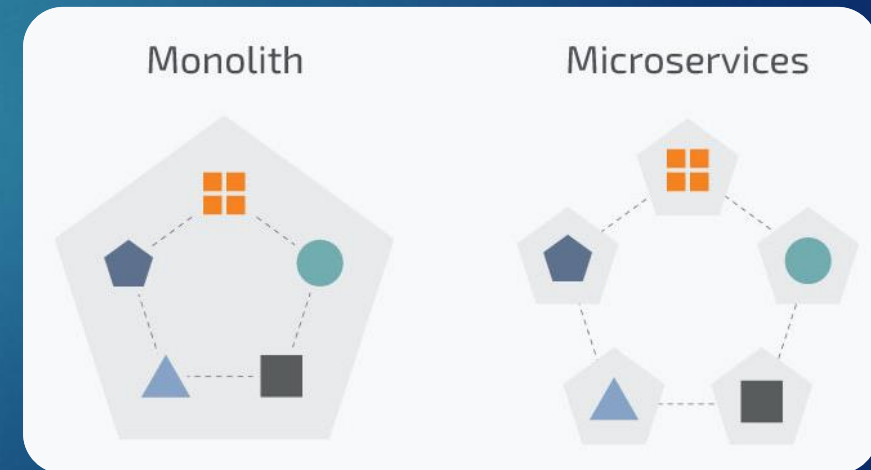
► Cons

- Lambda functions have resource limits, such as execution time limits (e.g., 15 minutes) and memory limits
- Debugging and local testing of Lambda functions can be more challenging compared to traditional development environments
- Developing extensively using Lambda and AWS services can lead to vendor lock-in, making it challenging to migrate to another cloud provider



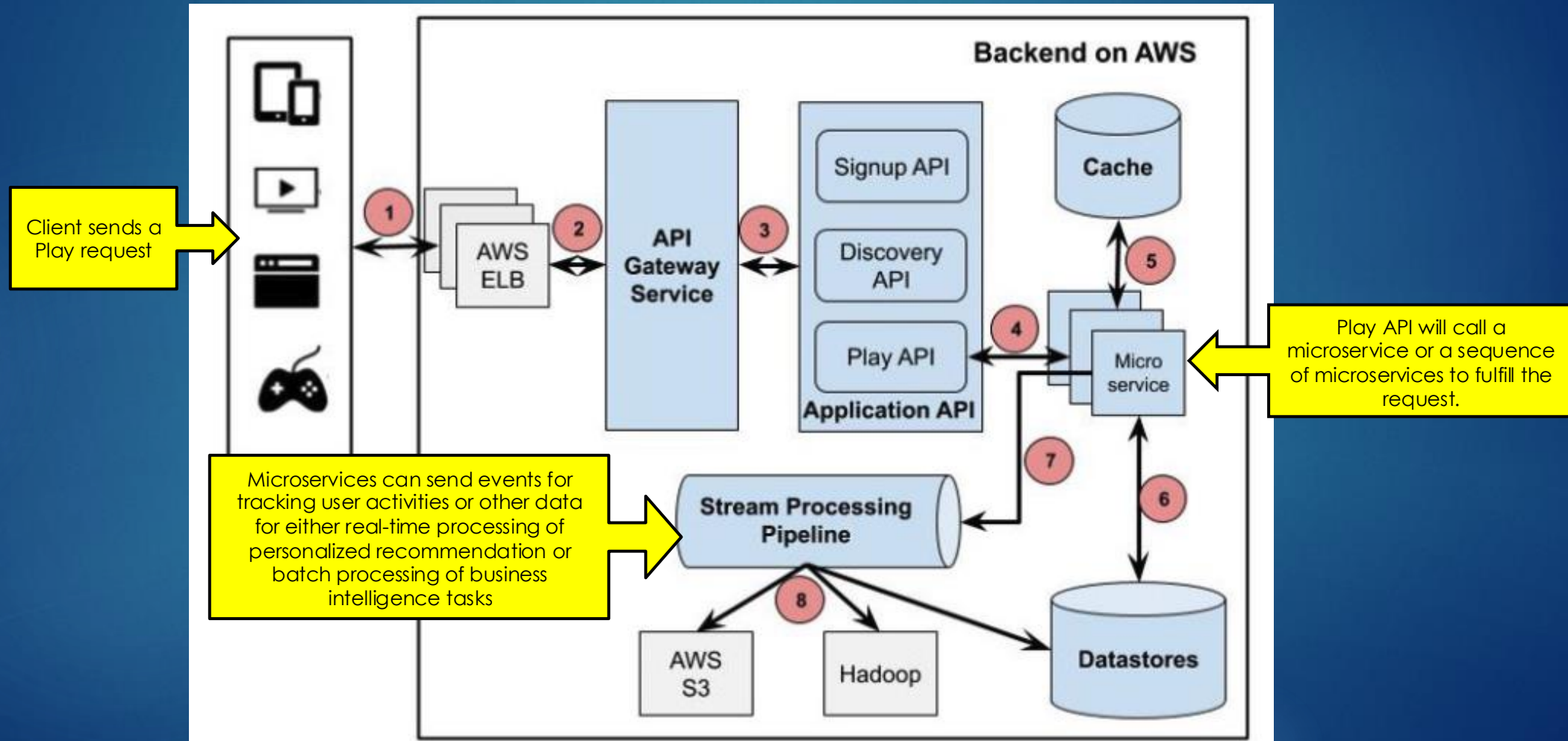
Microservices – Considerations

- ▶ Microservices architecture reduces the team management complexity, but it cannot diminish the need for team communication
 - ▶ Developers need to make sure that updates in one team's service does not break another's team functionality
- ▶ Although you can choose a different technology stack for different components, it can lead to a problem of non-uniform application design and architecture and possibly increase maintenance costs in the long run
- ▶ Inter-service communication needs to be secured to avoid any security breach. Due to several moving parts, these applications are more prone to security vulnerabilities.
- ▶ Testing of such applications can be more complex in comparison to monolith applications
- ▶ Microservices, when implemented incorrectly, can make poorly written applications even more dysfunctional



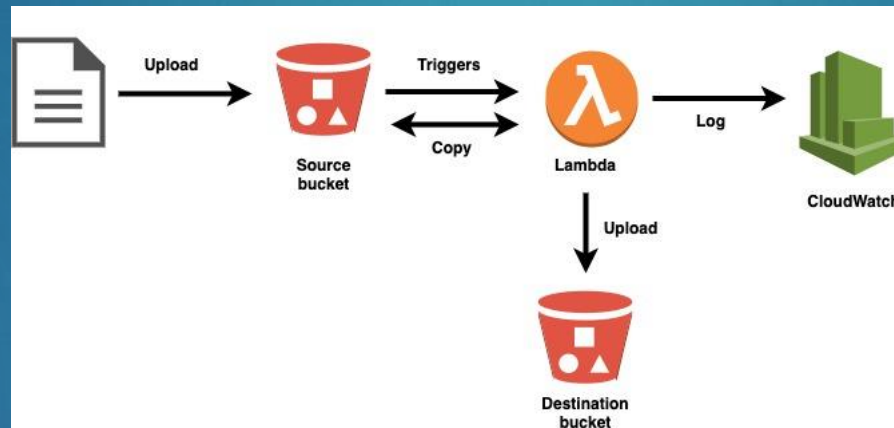
Netflix - Microservices

- ▶ Today, Netflix handles about two billion API requests every day



Create a Lambda Function with S3

- ▶ This activity will monitor an S3 bucket and copy it to another S3 bucket
- ▶ We will verify the message through CloudWatch, which is a monitoring tool that allows you to set alarms, visualize logs, metrics and many other related activities. We will explore CloudWatch later.



- ▶ Do the activity in **Assignments > Activity – Create Lambda Function with S3**
- ▶ There are 2 deliverables for this activity:
 1. Screenshot of the above activity
 2. Creating a “hello-world” Lambda function that is deployed to an API Gateway